GLOOBAL · ENGINEERING

End-of-Day Engineering Report

Date 2 September 2026 **Repository** GloobalV3 · main **Start of day** f89de31 **End of day** 3fbdae7 (pushed)

Deployed No

> This report covers **2 September 2026 only**. Earlier work appears solely where it is needed to explain something examined today, and is marked *[historical context]*.

1 · Executive Summary

7	2	5
DEFECTS FOUND	FIXED & MERGED	STILL OPEN
36/36	**215/215**	**0**
SERVER PAYMENTS CORRECT	DOMAIN TESTS	DEPLOYMENTS

What we investigated

A full audit of every payment, amount and currency path, opened in response to a report that some payments showed incorrect amounts or currencies, some were correct, and some appeared to behave differently before and after a re-login. The brief was to determine *whether* this was one root cause or several, and whether any of it was genuinely intermittent — not to assume.

What we found

Seven distinct defects in three groups. The most important finding is that **the server is correct**: 36 payments were traced end to end across six currency corridors in both amount bases, three runs each, and all 36 were arithmetically correct with every repeat byte-identical. The defects live almost entirely in the client display layer, in two server response projections, and in one client session-storage condition.

Exactly one defect was proven intermittent, and its trigger is deterministic given the input — it is not random.

What we fixed

REF	FIX	COMMIT	STATUS
RC-3	A repeated `idempotencyKey` now returns the same canonical payment result the original returned	`2f450a5`	**FIXED TODAY** merged, pushed
Session	A fresh registration keeps the bearer token it was just issued	`2d05fba`	**FIXED TODAY** merged, pushed

What remains unresolved

RC-1, RC-2, RC-4, RC-5 and a separate idempotency gap around client-initiated retries. All found today, none fixed today, all documented with reproduction evidence.

Overall status

> Two fixes verified and merged to `main`. **No money was found to move incorrectly at any point in the audit** — the open defects concern what is *displayed* and what is *recorded for later reading*, with two exceptions noted under RC-4.

> **Nothing shipped today.** Netlify and Render are still running the pre-fix build. Both fixes are inert in production until a manual deploy.

2 · Detailed Work Completed Today

2.1 · Payment / amount / currency audit

Task. Trace every way money can be initiated, every place an amount is displayed, and the contract carrying a payment from the amount box to the stored record and back out to history and receipts.

Method — five independent lines of evidence, all run today:

1. **Server contract trace.** The real `server.js` against a throwaway database on the same Atlas cluster, seeded deterministic FX (USD 1 · INR 95 · GBP 0.79 · JPY 160), executing a 36-payment matrix: 6 corridors × 2 amount bases × 3 runs. Captured the response body, both balances, the stored row and metadata, the settlement, the share leg, the receipts and both history projections.
2. **Client display trace.** The shipped helpers — `convert`, `fmt`, `currencyDecimals`, `buildTransactionSnapshot`, `buildHistoryReceipt`, `sumHistoryAmount` — concatenated exactly as `build_app.mjs` concatenates them and executed against the server's own recorded responses.
3. **Live FX comparison** against the table the server settles on (`open.er-api.com`, 2026-09-02 00:02 UTC).
4. **Live database inspection, read-only.** No write of any kind.
5. **Deployment check.** Local build output vs what Netlify serves; Render probed for an endpoint unique to the then-current commit.

Outcome. Seven defects plus a definitive exclusion of four hypotheses. Written up as `docs/audits/PAYMENT_AMOUNT_CURRENCY_AUDIT_2026-09-02.md` (712 lines), committed today as `bd5b1ab`. The audit changed no application code.

2.2 · RC-3 — idempotent duplicate response FIXED TODAY

SYMPTOM	A payment retried with the same `idempotencyKey` produced a different client-side record than the same payment on a first attempt.
ROOT CAUSE	Both duplicate paths answered with `cleanTransactionPayload` alone — the destination side of the payment only. Eighteen fields the 201 carries were absent.
FILES	`server/server.js` · `frontend/App.jsx` · `server/tests/idempotent-duplicate-response.test.mjs` (new) · `server/package.json`
FIX	A duplicate now answers with the same canonical result the original did, rebuilt from persisted documents. See §4.
WHY SAFE	Reads only — no balance, pool, ledger line, settlement, receipt or transaction is written. No FX lookup. Invents nothing: a pre-contract row answers `null`. The 201 path is unchanged except for sorting an array whose order was never read positionally.
TESTS	New regression suite **ALL CHECKS PASSED**; verified to fail on baseline; browser **12/12**; full server suite 87/90 with all three failures reproduced on the untouched baseline.
STATE	Commit `2f450a5` → merged `3fbdae7` → pushed. **Not deployed.**

2.3 · Registration / session token FIXED TODAY

SYMPTOM	A brand-new account reached the dashboard signed out. Every dashboard request went without an `Authorization` header and returned 401 — rendered as "*Unable to load balance*". Found while attempting browser verification of RC-3, which it blocked.
ROOT CAUSE	`sessionStore.js` stored the token as `(sameAccount && previous.token) || null`. At the first session save of a fresh registration the stored blob holds a token and **no user**, so `sameAccount` was false and the token was overwritten with `null`. See §5.
FILES	`backend/services/api/sessionStore.js` (+50/−3, three functional lines) · `financial-principles-tests/tests/registrationSessionToken.test.mjs` (new, 264 lines)
FIX	`hadPreviousAccount` — already present at the bottom of the same function — hoisted, and the token now reads the same answer.
WHY SAFE	The guard protects against account A's credential reaching account B, and that requires an account A. A blob with a token and no identity has no A to leak from: sign-out removes the whole key, and the token is only ever written by registration, login or passkey sign-in for the account saved moments later. A stored session naming a *different* account still yields no token — unchanged, and covered by tests.
TESTS	13/13 unit (9/13 fail on baseline); browser **42/42** across three countries, baseline reproducing the symptom in all three.
STATE	Commit `2d05fba` → merged `3fbdae7` → pushed. **Not deployed.**

2.4 · Other work performed today

- **Deployment consistency check.** Local `f89de31` built to `index-CtzF5nkc.js` / `index-5K8Tj4jh.css` — byte-identical filenames to what Netlify served — and Render answered an endpoint introduced in that commit. Deployment mismatch eliminated as a cause.
- **Read-only production data inspection.** Two scripts, no writes. Quantified RC-5 against real data.
- **Baseline reproduction of three pre-existing test failures**, so they could be attributed correctly rather than blamed on today's changes.
- **Documentation.** The audit report, committed as `bd5b1ab`.

3 · Payment Audit Findings From Today

FIXED TODAY | FOUND TODAY — NOT FIXED | LATENT | PRE-EXISTING | NOT VERIFIED

RC-1 · Client FX rate differs from the server's authoritative rate FOUND TODAY — NOT FIXED

The client prices every pre-confirmation figure from a static table in `backend/data/currencies.js`, whose own comment calls it "approximate, roughly-current figures for display purposes". The server settles on live rates via `server/lib/fxRates.js`. The payer is quoted one number and charged another.

PAIR	CLIENT STATIC	LIVE	CLIENT ERROR
INR→USD	0.010482	0.010526	−5.00%
USD→INR	95.398	95.001	+0.42%
INR→GBP	0.008347	0.007783	+28.5%
GBP→INR	119.80	128.48	−6.76%
INR→JPY	1.6549	1.6854	−2.10%
JPY→INR	0.6043	0.5933	+1.12%
USD→GBP	0.800	0.7394	+8.19%
EUR→INR	103.03	110.15	−6.46%

PAYMENT	SCREEN QUOTES	BALANCE ACTUALLY MOVES	GAP
INR payer → GBP payee, payee gets £1,000	₹119,802.33	~₹128,482.56	−₹8,680 (6.8% understated)
GBP payer → INR payee, payee gets ₹1,000	£8.35	~£7.78	+7.3% overstated
INR payer → USD payee, payee gets $1,000	₹95,398.15	~₹95,000.91	+0.42%

> **Observed live in the browser today.** During the RC-3 Playwright run the Send button quoted **₹1,198.02** for a 10 GBP payment; the server debited **₹1,202.53**.

Deterministic, not intermittent. The size of the error varies by corridor, which is what made it look inconsistent under manual testing.

RC-2 · The client records its own figure, not the server's FOUND TODAY — NOT FIXED

`handleRemoteSend` correctly returns the server's `debitAmount` and `senderCurrency`. Send Money never reads them: the local ledger post, the toast and the fresh receipt are all given the client-computed `senderAmount`. Scan & Pay prefers the server's figure for its history row but still passes the raw **destination** amount to `executeTransaction`, and labels its success toast with the payer's currency symbol on the payee's number.

> **Observed live today.** After the RC-3 browser payment the dashboard recorded **−₹1,198.02** — the client figure — for a payment that cost ₹1,202.53.

Deterministic. Reconciled away on the next profile read, which is what makes the dashboard balance appear to move on its own.

RC-3 · Idempotent duplicate response FIXED TODAY

See §2.2 and §4. This is the only defect proven intermittent today, and its trigger is deterministic given the input.

> **Correction made during the fix.** The audit initially stated that the client reaches this path routinely on a cold-start retry. Tracing the flow showed that is wrong: `httpClient` performs no automatic retry, and a user-initiated retry mints a fresh key. The 200-duplicate is reached by a genuine replay of the identical request — a double-submit, the concurrent index race, or a network-level replay. The defect and the fix stand; its frequency is lower than first stated. Recorded here rather than quietly dropped.

RC-4 · Creator Share currency metadata dropped FOUND TODAY — NOT FIXED LATENT

`server.js` passes `cashbackCurrency: senderCurrency` into `mintShareLegAndReceipts`. That function's parameter list does not include it, so it is silently dropped. The share row is then written with the **sender's** amount under the **destination** currency code.

CORRIDOR	PAYER REALLY GOT BACK	SHARE ROW SAYS	SHARE RECEIPTS SAY
INR→USD	1,900 INR	**1,900 USD**	1,900 USD
USD→INR	0.21 USD	**0.21 INR**	0.21 INR
INR→GBP	2,405.06 INR	**2,405.06 GBP**	2,405.06 GBP
GBP→INR	0.17 GBP	**0.17 INR**	0.17 INR
INR→JPY	118.75 INR	**118.75 JPY**	118.75 JPY
JPY→INR	337 JPY	**337 INR**	337 INR

The 201 response contradicts itself in one body: `cashback: 1900, cashbackCurrency: "INR"` alongside `shareTransaction: { amount: 1900, currency: "USD" }`. The client reads the second.

> **Not yet present in live data.** Of 125 `share` rows in production, all sampled are single-currency — no cross-currency payment to a Creator with a non-zero share rate has occurred. It will write bad records silently the first time one does.

Related, same function: `issueReceiptPair` writes one amount/currency pair to both parties' receipts, so the payer's stored receipt says `1000 USD` for a payment that cost ₹95,000. Nothing reads `Receipt` today, so this is stored-record integrity rather than a visible screen.

RC-5 · Dashboard `totalSent` sums the wrong side FOUND TODAY — NOT FIXED

`GET /api/transactions/:symbolId` sums `$amount`, the **destination** face value, so `totalSent` adds the counterparty's currency on every cross-border row as a bare number — then the dashboard renders it with the viewer's own symbol.

- Reproduction harness: a GBP account that really spent **£2,964.45** reported `totalSent` **363,759.48**, shown as GBP.
- **Live production data:** the busiest account reports **132,151.99 ₹** against **18,679.37 ₹** of actual debits — a **7× overstatement**.
- Browser run today: an INR account showed `totalSent: 10` after a 10 GBP payment.

`totalReceived` is correct in unit but is gross, before the payee's own Creator Share was withheld.

Retry with a new idempotency key can produce a second payment

FOUND TODAY — NOT FIXED NOT REPRODUCED

A user-initiated retry mints a **new** `idempotencyKey`. Past the server's 15-second identical-resend window, a timeout on a payment that actually committed would be seen as a fresh payment. Separate from RC-3 and not addressed by it.

Severity not rated: established by reading the flow. The scenario was **not** reproduced end to end today.

Registration / session token FIXED TODAY

Found today during RC-3 verification. See §2.3 and §5.

History reconstruction — fresh receipt ≠ reopened receipt FOUND TODAY — NOT FIXED

`mapServerTransaction` discards the year from an authoritative ISO timestamp at the client boundary, which is why the "2 Years" and "5 Years" history filters cannot reach past twelve months. `buildHistoryReceipt` re-derives the counterparty amount with the client rate instead of reading the stored destination amount the server already sent — a payment where the payee was credited exactly **£1,000** reopens as **£1,003.76**; **¥10,000** reopens as **¥9,825.72**. `ReceiptModal` recomputes the Creator Share instead of reading it, and formats it with no currency code.

QR payload carries no currency FOUND TODAY — NOT FIXED

`gloobalQR.js` encodes an ID, an amount in minor units and a checksum — no currency; the scanner infers it from the payee's registered country via a live lookup. Separately, `requestCents` multiplies by 100 unconditionally and the caption prints two decimals — wrong precision and a 100× waste of the payload range for a zero-decimal currency. A JPY request above ¥20,971 cannot be encoded at all.

Hypotheses tested and excluded today

HYPOTHESIS	VERDICT	EVIDENCE
Deployment mismatch	**Excluded**	Local `f89de31` build produced byte-identical asset filenames to what Netlify served; Render answered an endpoint unique to that commit
Service-worker / cached bundle	**Excluded**	`autoUpdate` with a NetworkFirst document handler — bounded to one reload, and the live bundle was current
Backend arithmetic	**Excluded**	36/36 payments correct, every repeat identical
Race in the payment write	**Excluded**	Conditional `$inc` debit, row created inside the same Mongo transaction, unique partial index on `(fromUserId, metadata.idempotencyKey)`
Genuinely random behaviour	**Excluded**	One `Math.random()` reaches a financial display, but `mapServerTransaction` sets `shareRate: 0` rather than leaving it undefined, so no server-restored row can reach it

Pre-existing conditions confirmed today PRE-EXISTING

ITEM	STATUS
`concurrency-scale` failing under Atlas contention	Reproduced on the untouched baseline
`geu-concurrency` failing under Atlas contention	Reproduced on the untouched baseline
`geu-invariants` failing because the live FX provider is reachable	5 failures on baseline and with the fix — identical
`scan-undeclared` reporting `Notification`	Documented non-failure in `CLAUDE.md`
`probe-screens` two SSR-only failures	Documented in `CLAUDE.md`
Auto-deploy broken on Render and Netlify	*[historical context]* documented in `CLAUDE.md`

4 · RC-3 — Detailed Fix

The problem

Two paths recognise an already-executed request: a pre-check that finds a committed row for the same `(fromUserId, idempotencyKey)`, and a recovery path for the loser of the unique-index race. Both answered:

```
{ success, duplicate: true, message, transaction: cleanTransactionPayload(...) }
```

Why `cleanTransactionPayload` was insufficient

It is not a summary of a payment — it is the **destination side** of one, by design. `amount` and `currency` on a `Transaction` row are the receiver's face value; the sender's own side lives in `metadata`.

The client reads `debitAmount` to decide what a payment cost. Its fallback was the typed amount — the receiver's figure on a cross-border payment. Both halves stayed honestly labelled, which makes the defect subtler than a mislabel and worse: **the row switched sides**. The same payment recorded either "₹95,000 out" or "$1,000 out" depending on which response the browser received.

Measured today, before the fix

```
FIRST  201  sourceAmount 95000 INR | destinationAmount 1000 USD | debitAmount 95000
            fxRate 95 | cashback 1900 INR | payeeReceives 980
            settlement + share leg + 4 receipts

RETRY  200  { duplicate: true }
            sourceAmount, sourceCurrency, destinationAmount, destinationCurrency,
            debitAmount, senderCurrency, fxRate, fxRateSource, amountBasis,
            cashback, cashbackCurrency, cashbackRate, payeeReceives, newBalance,
            settlement, shareTransaction, receipts, assetSeed  — ALL ABSENT
```

What the client derived from that: `debitAmount → null`, `senderCurrency → null`, `cashback → 0`, `cashbackRate → 0`, `shareTransactionId → ""`.

What is returned now

SOURCE DOCUMENT	FIELDS TAKEN
`Transaction` + `metadata`	`sourceAmount`, `sourceCurrency`, `destinationAmount`, `destinationCurrency`, `debitAmount`, `senderCurrency`, `fxRate`, `fxRateSource`, `amountBasis`, `cashback`, `cashbackCurrency`, `cashbackRate`, and the `transaction` projection
`LedgerEntry` (receiver's credit line)	`payeeReceives` — read as what actually moved, not re-derived as amount minus cashback
`Settlement`	the full settlement projection, both sides plus rate and status
`Transaction` (`type: 'share'`)	`shareTransaction` — reference, amount, currency
`Receipt` (payment + share legs)	`receipts`, sorted by leg then role
`AssetSeed`	`assetSeed` via `computeSeed`
`User.balance`	`newBalance`, rounded the way the 201 rounds it

Why it is deterministic, writes nothing, and computes no FX

- **No FX lookup.** Every currency and rate comes off the stored row. A rate fetched now is not the rate the payment settled at.
- **No writes.** Only `find` / `findOne` / `findById`.
- **No invention.** A row predating a field answers `null` — the same rule the history projection follows.

- **Stable ordering.** The four receipts are written by two concurrent pairs, so their order is not guaranteed. Both responses now sort by leg then role.
- **Two fields are deliberately as-of-now**, exactly as the 201 reports them: `newBalance`, and the time-accruing half of `assetSeed`. Their recorded halves do not move.

Test results

Server regression suite — 417 lines, **ALL CHECKS PASSED**. Covers INR→USD, USD→INR, INR→GBP, GBP→INR in both amount bases; the same-currency and no-Creator-Share shapes; three retries per cell compared field for field; the concurrent-race path; and exactly-once across transaction, share leg, settlement, ledger lines, receipts, history rows and both balances.

Verified to fail on the untouched baseline — 21 FAILs per retry, every compared field `duplicate=undefined`.

Browser verification — real bundle, local API, `page.route` performing the send twice and handing the app the **duplicate** response. **12/12 passed**:

```
first response  : sourceAmount=1202.53 INR debitAmount=1202.53 destination=10 GBP
delivered (retry): sourceAmount=1202.53 INR debitAmount=1202.53 destination=10 GBP  duplicate=true

PASS  the debit was NOT replaced by the destination amount — debit=1202.53 destination=10
PASS  the receipt does not show the destination figure wearing a rupee sign
PASS  exactly one Transaction row for this payment
PASS  the account has exactly one send in total
```

Commit `2f450a5` — 4 files, +651/−19. Merged in `3fbdae7`, pushed. Not deployed.

5 · Registration / Session Token Fix

The flow that discarded the token

1. `POST /api/register-symbol` answers with a bearer token.
2. `GloobalApi.register` stores it through `gloobalAuthTokenSave`, which writes `{ savedAt, token }` and **no** `user` — no session has been saved yet.
3. `POST /api/pin/set` succeeds, using that token.
4. The biometric step calls `saveSession(user, …)` — the **first** `gloobalSessionSave` of this account's life.

At step 4, `previousUser` is `undefined`, so `sameAccount` is false, and `token: (sameAccount && previous.token) || null` overwrote a valid credential with `null`.

Why the first dashboard request returned 401

`gloobalApiRequest` attaches `Authorization` only when a token exists. With it nulled, profile, transactions, assets, PayLater and interest were all sent anonymously. Captured today on the baseline build, across three countries:

```
[net] 200 auth=Bearer eyJzdWIiOiI2… /api/profile/…   Profile updated successfully
[net] 401 auth=(none)               /api/profile/…   Sign in to continue.
[net] 401 auth=(none)               /api/transactions/…?type=all
[net] 401 auth=(none)               /api/assets/…
```

A first-ever sign-in on a device with empty storage hit the same line for the same reason.

The corrected condition

```
const hadPreviousAccount = Boolean(previousUser && previousUser.symbolId);
…
token: (((sameAccount || !hadPreviousAccount) && previous && previous.token) || null),
```

`hadPreviousAccount` already existed at the bottom of the same function, where the account-switch notice draws exactly this distinction; it is hoisted so the token reads the same answer. Three functional lines changed.

Why account A's token cannot leak into account B

- `gloobalSessionClear` removes the whole storage key on sign-out — token and identity together — so nothing survives for a later account to pick up.
- `gloobalAuthTokenSave` is only ever called by registration, login and passkey sign-in, each for the account whose session is saved moments later. A token sitting next to no identity is, by construction, that account's own.
- A stored session that **names** a different account still yields no token. Untouched, and covered by two tests.

Remaining edge case, deliberately not changed STILL PRESENT

> If account B signs in while A's session is still stored — without A signing out first — B's own freshly-minted token is discarded, because `gloobalAuthTokenSave` always runs before `gloobalSessionSave`. B is asked to sign in again; nothing leaks. Widening the condition would weaken a deliberate protection, and it is outside the reported bug. **No UI path was found that reaches it** — signing in normally follows sign-out, which clears everything. Not fixed, and not independently verified as reachable.

Regression coverage added — 13 tests, all passing

Loads the real `sessionStore.js` in a `vm` with an in-memory `localStorage`, the way `build_app.mjs` loads it, so it exercises the shipped file rather than a fork and leaves `app_bundle_testonly.mjs` untouched.

a fresh registration keeps the token it was just issued	✓	a returning account's token is carried across a save that does not know it	✓
a fresh registration ends up with an authenticated session	✓	an ID rename keeps the session and its token	✓
the dashboard's first reads would carry a bearer token	✓	signing out clears the token	✓
no second sign-in is needed	✓	`clearAuthToken` drops the credential, keeps the identity	✓
a first-ever sign-in on empty storage keeps its token	✓	a different account signing in inherits nothing	✓
sign-out then a second account signing in is clean	✓	a real switch announces itself; a first registration does not	✓
no stored session at all still saves cleanly	✓		

Test results

- Unit **13/13 pass**; on the untouched baseline **9 of 13 fail**.
- `financial-principles-tests` full suite **215/215 pass**.
- Browser, three countries in clean contexts: **42/42 pass** — token stored, all six dashboard reads carried an `Authorization` header, zero 401s, profile 200, transactions 200, no "*Unable to load balance*", card denominated in the account's own currency, balance 10,000 on the authenticated read.
- Browser on the baseline build: **15 failures** — `token=null`, unauthenticated reads, 5 of 6 calls 401, and the literal reported symptom in all three countries.

> **Not verified in the browser:** the on-screen balance *figure* stays behind the app's privacy mask, and revealing it prompts for biometric — a UI gate unrelated to this fix. The amount was asserted where it arrives, in the authenticated profile response the card is built from.

Commit `2d05fba` — 2 files, +314/−3. Merged in `3fbdae7`, pushed. Not deployed.

6 · Testing Evidence

All results below were produced today.

6.1 · New tests written today

TEST / SUITE	RESULT	PASSED	FAILED	NOTE
`server/tests/idempotent-duplicate-response.test.mjs`	PASS	all	0	ALL CHEC PASSI corric × 2 b × 3 re + san curre no-sh + concu race
— same suite, on untouched baseline	FAIL (EXPECTED)	—	21 per retry	Confi it cat the d
`financial-principles-tests/tests/registrationSessionToken.test.mjs`	PASS	13	0	
— same suite, on untouched baseline	FAIL (EXPECTED)	4	9	Confi it cat the d

6.2 · Existing suites run today

TEST / SUITE	RESULT	PASSED	FAILED	NOTES
`financial-principles-tests` (full)	PASS	215	0	Includes the 13 new tests
`server` full suite (`npm test`)	PARTIAL	87	3	All 3 failures pre-existing — see 6.4
`server/tests/auth-and-access`	PASS	all	0	ALL CHECKS PASSED
`server/tests/cross-currency-transfer`	PASS	all	0	
`server/tests/merchant-share-flow`	PASS	all	0	
`server/tests/corridor-currency-integrity`	PASS	all	0	
`server/tests/transfer-atomicity`	PASS	all	0	
`server/tests/hardening-fixes`	PASS	all	0	
`tools/frontend/probe-panels.mjs`	PASS	3	0	3/3 panels render
`tools/frontend/scan-undeclared.mjs`	PASS (DOCUMENTED)	—	—	Only `Notification` — a documented non-failure
`tools/frontend/probe-screens.mjs`	PASS (DOCUMENTED)	—	2	Both SSR-only, documented in `CLAUDE.md`
`node build_app.mjs`	PASS	—	—	Bundle rebuilt clean after every change
`node --check` server.js · sessionStore.js	PASS	—	—	

6.3 · Browser (Playwright) runs today

RUN	RESULT	PASSED	FAILED	NOTES
RC-3 duplicate-response drive	PASS	12	0	Real bundle, local API, duplicate response delivered to the app
Fresh registration, 3 countries	PASS	42	0	India / United States / United Kingdom, clean context each
Fresh registration on baseline build	FAIL (EXPECTED)	—	15	Reproduces the reported symptom in all three countries
Audit-phase exploratory drive	INCOMPLETE	—	—	Reached Send Money; did not complete a payment

6.4 · Pre-existing failures — reproduced on the untouched baseline

> **No new failures were introduced today.** Every suite that passed before the changes passes after them. Each failure below was confirmed by stashing the day's changes and re-running.

SUITE	FAILURE	BASELINE BEHAVIOUR
`concurrency-scale`	"no unexpected status codes" — 500s instead of 400s	Sampled 3× on baseline: 1 pass, 2 fail. Mongo write conflicts under 100-way contention on shared Atlas. Financial invariants still hold — balance lands on exactly 0, receiver credited exactly 5000, money conserved, 50 success rows, 100 ledger lines, nothing stranded
`geu-concurrency`	Same class — 500s, accepted counts short	Fails on baseline
`geu-invariants`	5 checks about stale/captured FX rates	**5 failures on baseline and 5 with the fix — identical.** The live FX provider is reachable from this machine, so the test's "stale cached rate" assumptions do not hold

6.5 · Explicitly not verified

ITEM	WHY
Full UI-driven payment matrix across all six corridors	The audit-phase browser drive reached Send Money but did not complete a payment; the matrix was completed at the API and display-function layers. Browser coverage of a payment is one corridor (INR→GBP, destination basis)
The double-payment scenario	Established by reading the flow; not executed
The account-switch edge case	No UI path found that reaches it; not independently exercised
Render's exact running commit	Cannot be probed unauthenticated for the RC-3 change — see §10
On-screen balance figure after registration	Sits behind the app's privacy mask and a biometric prompt

7 · Issues Still Open

Everything below was found today and is **not fixed**.

ISSUE	SEVERITY	ROOT CAUSE	IMPACT	NEXT STEP
RC-1 Client FX ≠ server FX	High *supported by measurement*	Established — static `RATES` table vs live `fxRates.js`	Payer confirms a price up to ~7% away from what leaves their balance. No money moves incorrectly	Serve the server's live rate to the client and quote from it; leave tl static table as a labell offline estimate. Drop `convert()` 's hardco 2-dp rounding
RC-2 Client records its own figure	High	Established — Send Money discards `debitAmount` / `senderCurrency`; Scan & Pay passes the destination amount to the local ledger	Receipt, local ledger and dashboard disagree with the money until the next profile read corrects them	Read the server's figu for the ledger post, tc and receipt; pass the settled source amour Scan & Pay; fix its toa currency
RC-4 Creator Share currency dropped	High, **latent**	Established — `cashbackCurrency` passed but absent from the parameter list	The first cross-currency payment to a Creator with a share rate will silently write a sender-currency amount under the destination currency code	Add the parameter; c it onto the share row and receipts; split `issueReceiptPair` s each party's copy carı its own side
RC-5 Dashboard `totalSent`	High *wrong on live data*	Established — the aggregate sums destination-denominated `$amount`	The headline PAID figure	Sum `metadata.debitAmou` return the account's c

ISSUE	SEVERITY	ROOT CAUSE	IMPACT	NEXT STEP
			is wrong for any account with cross-border sends (7× on the busiest live account)	currency, report rows lacking it as an excluc count
Retry with a new idempotency key	NOT RATED flow reading only	Established by inspection — a retry mints a fresh key, so only the 15-second window stands	Potentially a duplicate real payment after a timeout. **Not reproduced**	Reproduce first. If confirmed, reuse the across a retry of the same authorised payment
History reconstruction fresh ≠ reopened receipt	Medium	Established — the year is discarded at the client boundary; `buildHistoryReceipt` re-converts; `ReceiptModal` recomputes	Same payment shows different figures before and after a reload; 2- and 5-year filters cannot reach past twelve months	Carry the ISO timesta through and filter on read the stored counterparty amount and share amount
QR payload no currency, minor units	Medium	Established	A zero-decimal currency cannot express a request above ~20,971 units; the caption states a	Scale `requestCents` the requester's own `currencyDecimals`; name the currency in ceiling message

ISSUE	SEVERITY	ROOT CAUSE	IMPACT	NEXT STEP
			precision it does not have	
Session account switch without sign-out	Low degraded, not a leak	Established	B is asked to sign in again. No UI path found that reaches it	Leave unless a reacha path is found

8 · Today's Timeline

Timestamps appear only where reliable evidence exists. Git commit times are exact; surrounding phases are ordered from the session record without invented times.

– **Audit opened.** Repository recon; payment routes, models, FX and settlement libraries read

– Server payment contract traced end to end; 36-payment matrix executed against a throwaway database

– Idempotent-retry response shape captured — the RC-3 evidence

– Client display helpers executed against the server's recorded responses; live FX table fetched and compared

– Read-only inspection of live transaction data; RC-5 quantified against production

– Deployment consistency verified — both targets on `f89de31`; deployment excluded as a cause

– Exploratory browser drive reached Send Money; did not complete a payment

– Audit report written

– **RC-3 work begins.** Branch created; flow traced; reachability claim corrected

– `duplicatePaymentResponse` implemented; both duplicate sites and receipts ordering updated; Scan & Pay fallback narrowed

– Regression suite written; passed; verified to fail on the stashed baseline

– Full server suite run (87/90); the 3 failures reproduced on the untouched baseline and attributed as pre-existing

– Browser verification of RC-3: 12/12

`19:06:09` **Commit** `2f450a5` — fix: return canonical result for idempotent payments

– **Session-token work begins.** Branch created off the RC-3 branch; registration flow traced

– Condition corrected; 13-test regression suite written; passed; 9/13 fail on the stashed baseline

– Browser verification across three countries: 42/42; baseline run reproduces the symptom (15 failures)

– `financial-principles-tests` 215/215; `auth-and-access` passed; probes clean

`20:01:15` **Commit** `2d05fba` — fix: keep the token a fresh registration was just issued

`20:02:25` **Commit** `bd5b1ab` — docs: payment amount/currency audit

`20:02:35` **Merge** `3fbdae7` into `main` (`--no-ff`)

– Pre-push verification on `main`: 215/215, RC-3 suite ALL CHECKS PASSED, syntax and bundle clean

– **Pushed** `f89de31..3fbdae7` to `origin/main`. No deployment triggered

9 · Git / Change Summary

Branches used today

BRANCH	PURPOSE	STATE
`fix/rc3-idempotent-duplicate-response`	RC-3	merged, still present locally
`fix/registration-session-token`	session token + docs; stacked on the RC-3 branch	merged, still present locally
`main`	integration	pushed to `origin/main`

Commits created today

SHA	TIME (IST)	SUBJECT
`2f450a5`	19:06:09	fix: return canonical result for idempotent payments
`2d05fba`	20:01:15	fix: keep the token a fresh registration was just issued
`bd5b1ab`	20:02:25	docs: payment amount/currency audit, 2 September 2026
`3fbdae7`	20:02:35	Merge fix/registration-session-token into main

Files changed today (`f89de31..3fbdae7`) — 7 files, +1,677 / −22

FILE	CHANGE	BELONGS TO
`server/server.js`	+206	RC-3
`frontend/App.jsx`	+44	RC-3
`server/package.json`	+3/−1	RC-3 (test wiring)
`server/tests/idempotent-duplicate-response.test.mjs`	new, 417	RC-3
`backend/services/api/sessionStore.js`	+53/−3	session token
`financial-principles-tests/tests/registrationSessionToken.test.mjs`	new, 264	session token
`docs/audits/PAYMENT_AMOUNT_CURRENCY_AUDIT_2026-09-02.md`	new, 712	audit

State of each change

CHANGE	UNCOMMITTED	COMMITTED	PUSHED	MERGED	DEPLOYED
RC-3 fix	—	✅	✅	✅	❌
Session-token fix	—	✅	✅	✅	❌
Audit report	—	✅	✅	✅	n/a

Working tree at end of day: **clean**, apart from the report artifacts generated by this reporting task.

10 · Deployment Status

STAGE	STATE	EVIDENCE
Local	UP TO DATE	Working tree clean at `3fbdae7`
Committed	YES	Four commits, listed in §9
Pushed	YES	`f89de31..3fbdae7 main -> main`; local and remote both at `3fbdae7`, 0 ahead / 0 behind after re-fetch
Merged to `main`	YES	Merge commit `3fbdae7`
Deployed to Netlify	NOT DEPLOYED	Netlify serves `/assets/index-CtzF5nkc.js`. A clean production build of `3fbdae7` produces `index-BVnhr67M.js`. The served bundle is the `f89de31` build verified this morning
Deployed to Render	NO DEPLOY TRIGGERED COMMIT NOT VERIFIED	No manual deploy was triggered today. Auto-deploy is documented as broken *[historical context: `CLAUDE.md` — Render's GitHub App lost repository access at the rename]*. The RC-3 change cannot be probed unauthenticated, so Render's exact commit could not be confirmed

> **Both fixes have no effect in production yet.** The session-token fix is entirely client-side and needs a Netlify deploy; RC-3 changes `server/` and needs a manual Render deploy.

11 · Recommended Next Work

1. **RC-4 — Creator Share currency.** Smallest change, and the only open item that will silently write permanently wrong records. Do it before any Creator is paid across a corridor.
2. **RC-2 — stop discarding the server's figures.** Makes the receipt, the local ledger and the dashboard agree with the money, and is a prerequisite for the right shape of the RC-1 fix.
3. **RC-1 — one rate.** With RC-2 done, the client rate only affects the quote, which is the right place to fix it properly.
4. **RC-5 — dashboard totals.** Wrong on live data today; a contained change to one aggregate.

5. **Reproduce the new-idempotency-key retry scenario**, then decide. Do not fix on an unreproduced hypothesis.
6. **History and receipt reconstruction.** Removes the before/after-reload difference and repairs the long history filters.
7. **QR minor units.** Contained; the payload currency question is a separate product decision.

> **Deployment decision, separate from the above:** both merged fixes are inert until Netlify and Render are deployed manually. That is a call for the founder, not a code change.

Prepared 2 September 2026. All figures were produced by runs performed today; where something was not verified, it says so. Companion document: `docs/audits/PAYMENT_AMOUNT_CURRENCY_AUDIT_2026-09-02.md`.